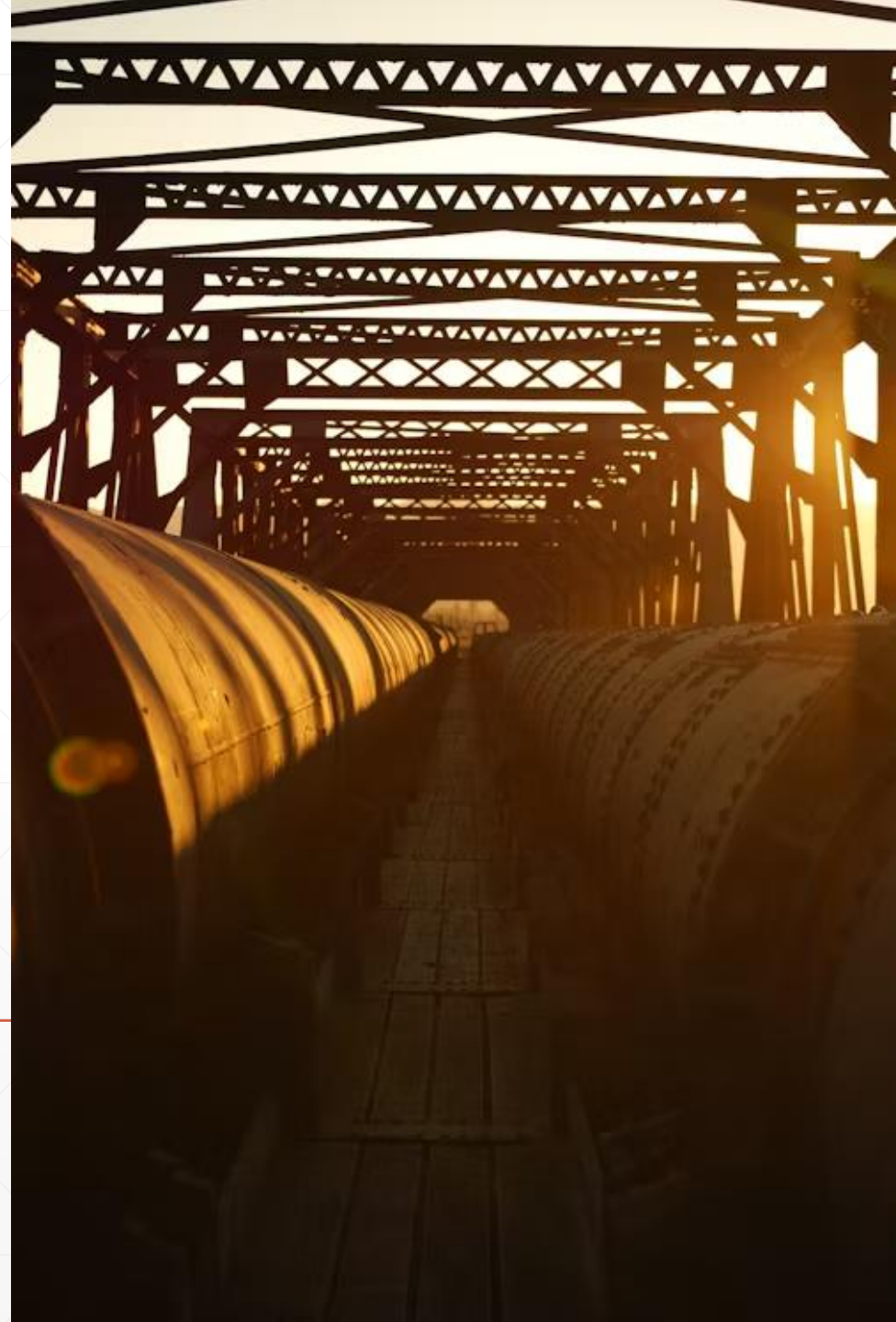
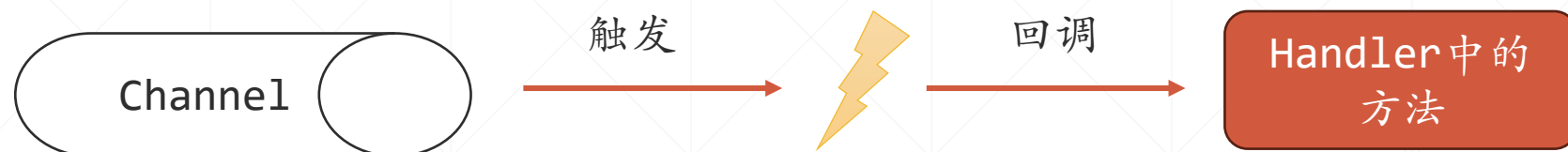


Channel Pipeline

北京理工大学计算机学院
金旭亮



复习



连接、断开、数据到达.....

在实际开发场景中，对于收到的数据，往往需要进行多个步骤的处理，将这些处理集中于单个Handler中完成是不合理的，存在许多弊端，比如难以重用，难以测试，难以维护等等。

如果将整个处理工作分解为多个步骤，每个步骤都用一个Handler处理，每个Handler都将自己工作的结果，传给下一个Handler，这个就能解决上面提到的问题，Netty为此设计了一个Pipeline来将多个Handler装配为一条“数据处理流水线”。

Channel与ChannelPipeline



一个客户端连接对应着一个Channel，这个Channel的所有处理逻辑，由多个ChannelHandler承载，这些Handler前后相连，放在一个ChannelPipeline对象里。



ChannelPipeline本质上是一个双向链表，每个链表节点如下所示，可以看到，每个Handler都配套有一个上下文对象（ChannelHandlerContext），正是通过这个对象让信息可以在多个Handler中传送。



入站的消息，会在InboundHandler中传递，对应地，出站的消息，则会在OutboundHandler中传递。

详解ChannelHandlerContext



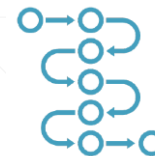
ChannelHandlerContext代表了ChannelHandler和ChannelPipeline之间的关联，每当有ChannelHandler添加到ChannelPipeline中时，都会创建 ChannelHandlerContext。ChannelHandlerContext的主要功能是管理它所关联的ChannelHandler和在同一个ChannelPipeline中的其他 ChannelHandler之间的交互。



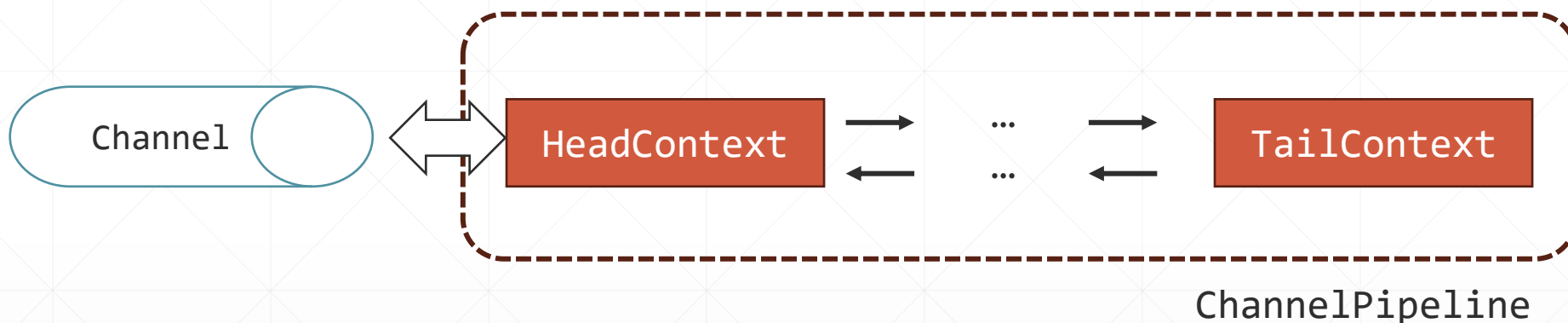
ChannelHandlerContext有很多的方法，其中一些方法也同时存在于Channel和ChannelPipeline中，但是有一点重要的不同：

- 如果调用Channel或者ChannelPipeline上的这些方法，它们将沿着整个ChannelPipeline进行传播。
- 而调用位于ChannelHandlerContext上的相同方法，则将从当前所关联的ChannelHandler开始，并且只会传播给位于该ChannelPipeline中的下一个能够处理该事件的ChannelHandler。

ChannelPipeline与双向链表



多个HandlerContext（关联着Handler和Channel）首尾相接，构成一个双向链表，Netty会自动在链表前后添加两个特殊的Handler，代表链表头与尾，完成一些基础性的工作，通常情况下，可以不必理会这两个“特殊的”Handler。

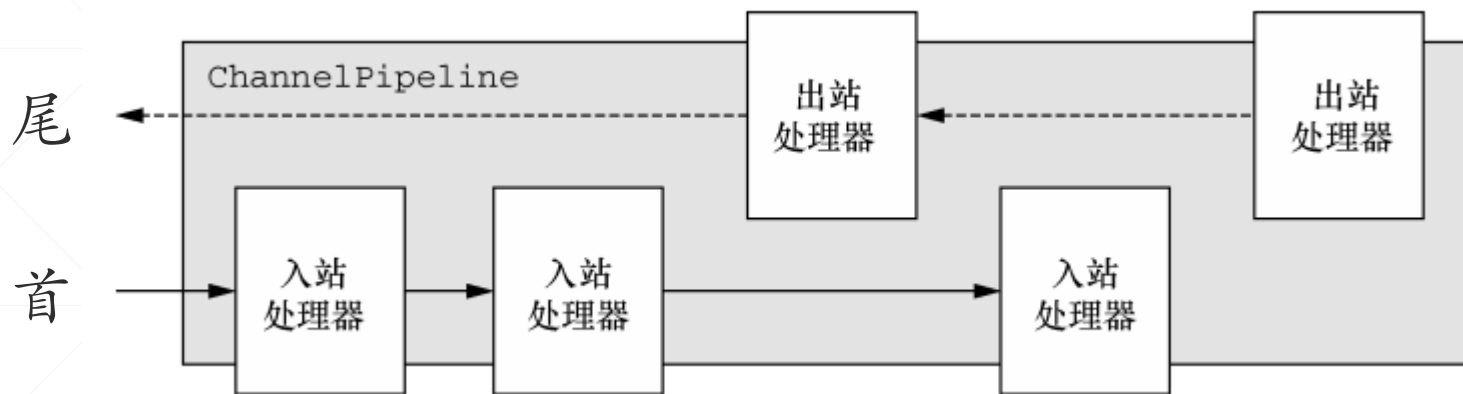


消息从前一个Handler传到下一个Handler，是通过调用fireChannelRead()方法实现的，这个方法，Channel/ChannelPipeline和ChannelHandlerContext都有，前者让消息在整条流水线中传播，而后者，则是从“当前位置”开始，向后传播。



fireChannelRead()方法本质上是一个循环，遍历流水线，找出下一个可以接收消息的Handler。如果不调用此方法，则消息将会停止在这个Handler，不再向后传播。

管线中Handler的连接方式



Netty总是把入站Handler作为头部，而将出站Handler作为尾端。



在ChannelPipeline传播消息时，它会检查管线中的下一个ChannelHandler的类型是否和当前消息的传播方向相一致。如果不一致，ChannelPipeline将跳过该ChannelHandler并前进到下一个，直到它找到和该事件所期望的方向相匹配的为止。由此形成了两条泾渭分明的“入站”和“出站”路径。



Pipeline中各个Handler排列的顺序是比较重要的，开发时要小心，推荐将各出站处理器按照数据流动方向“逆向”排列（因为Netty是将信息从队尾发到队首），统一放到pipeline前半段。

客户端Handler

```
public class MySimpleClientHandler extends ChannelInboundHandlerAdapter {  
  
    @Override  
    public void channelActive(ChannelHandlerContext ctx) throws Exception {  
        String info = LocalTime.now() + " Hello!";  
        ByteBuf byteBuf = Unpooled.wrappedBuffer(info.getBytes());  
        ctx.writeAndFlush(byteBuf);  
        System.out.println("发消息给Server:" + info);  
    }  
  
    @Override  
    public void channelRead(ChannelHandlerContext ctx, Object msg) throws Exception {  
        ByteBuf byteBuf = (ByteBuf) msg;  
        System.out.println("收到Server传来的消息: " + byteBuf.toString(StandardCharsets.UTF_8));  
    }  
}
```

本讲示例中的客户端Handler，在连接上服务器时，发送一条“Hello!”消息给客户端，然后再接收从服务端传回的信息。

客户端主程序

```
public class MyNettyClient {  
    public static void main(String[] args) throws InterruptedException {  
        NioEventLoopGroup workerGroup = new NioEventLoopGroup();  
        Bootstrap bootstrap = new Bootstrap();  
        bootstrap.group(workerGroup)  
            .channel(NioSocketChannel.class)  
            .handler(new MySimpleClientHandler());  
        var connectFuture = bootstrap.connect("127.0.0.1", 9000)  
            .addListener(future -> {  
                if (future.isSuccess()) {  
                    System.out.println("连接成功");  
                } else {  
                    System.out.println("连接失败");  
                }  
            });  
        connectFuture.channel().closeFuture().sync();  
    }  
}
```

由于客户端只有一个Handler，所以，在启动时直接调用handler()方法设定使用前页PPT所展示的Handler。

另外，由于客户端在发送信息后，还需要接收服务端传回的信息，为此，程序中没有关闭线程池，需要在IntelliJ中手动点击“关闭”按钮结束客户端程序的运行。

服务端定义的入站Handler

为了展示ChannelPipeline的特性，服务端一共定义了6个Handler，3个入站3个出站。这些Handler都是高度类似的，只是输出的信息不一样。

以下是一个入栈Handler的示例：

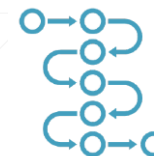
```
public static class SimpleInHandlerB extends ChannelInboundHandlerAdapter {  
    @Override  
    public void channelRead(ChannelHandlerContext ctx, Object msg) throws Exception {  
        ByteBuf buf = (ByteBuf) msg;  
        String info = buf.toString(StandardCharsets.UTF_8);  
        System.out.println("SimpleInHandlerB.channelRead():" + info);  
        ((ByteBuf) msg).resetReaderIndex();  
        super.channelRead(ctx, msg);  
    }  
}
```

这个参数，就是从前一个Handler传来的消息

注意这里调用了基类的同名方法，这个很重要，必不可少，没有这个方法，消息将无法传送。

示例中的3个入站Handler，分别为：SimpleInHandlerA、SimpleInHandlerB和SimpleInHandlerC。

构建ChannelPipeline



Channel的pipeline()方法，返回这个通道所关联的管线，调用这个管线的addLast()方法，可以将Handler加入到管线中。

```
//测试入站pipeline
private static void testInboundChannelPipeline(NioSocketChannel ch) {
    ch.pipeline().addLast(new MyServerPipeline.SimpleInHandlerA());
    ch.pipeline().addLast(new MyServerPipeline.SimpleInHandlerB());
    ch.pipeline().addLast(new MyServerPipeline.SimpleInHandlerC());
}
```



除了addLast(), ChannelPipeline还提供了addFirst, addBefore, addAfter三个方法在特定的位置加入Handler。



有加必有减，可以调用removeFirst, removeLast, replace方法删除或替换掉特定的Handler，从而“动态”地调整管线。

构建管线的关键组件

ChannelInitializer<C>		
  <code>initMap</code>	<code>Set<ChannelHandlerContext></code>	
  <code>logger</code>	<code>InternalLogger</code>	
  <code>handlerAdded</code> (<code>ChannelHandlerContext</code>)		<code>void</code>
  <code>channelRegistered</code> (<code>ChannelHandlerContext</code>)		<code>void</code>
  <code>removeState</code> (<code>ChannelHandlerContext</code>)		<code>void</code>
  <code>initChannel</code> (<code>ChannelHandlerContext</code>)		<code>boolean</code>
  <code>initChannel</code> (<code>C</code>)		<code>void</code>
  <code>exceptionCaught</code> (<code>ChannelHandlerContext</code> , <code>Throwable</code>)		<code>void</code>
  <code>handlerRemoved</code> (<code>ChannelHandlerContext</code>)		<code>void</code>

定义好了多个Handler之后，把它们装配为一条管线，关键的组件是ChannelInitializer这个抽象类，这个类定义有一个抽象方法：

```
protected abstract void initChannel(C ch) throws Exception;
```

这个方法的泛型参数C，代表一个派生自Channel的子类型。正是在这个方法中，我们可以写代码调用传入的通道对象的相关方法，完成管线的构建任务。

具体代码见下页PPT。

将管线与管道相关联

ServerBootstrap类的childHandler()方法负责为每个与客户端相连接的通道，关联一条管线，具体方式很简单，就是将一个ChannelInitializer对象作为参数传给它就行了。

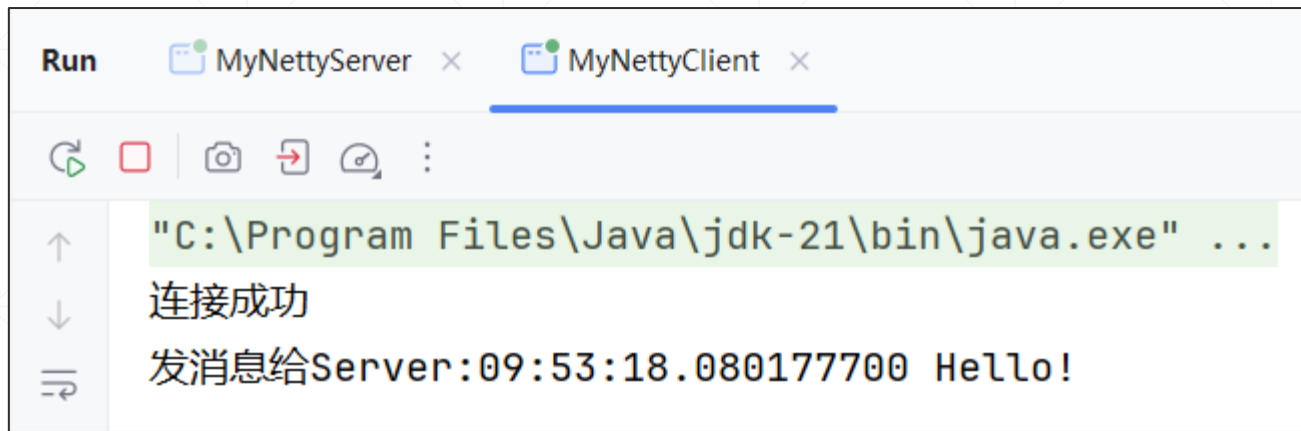
```
ServerBootstrap boot = new ServerBootstrap();

boot.group(bossGroup, workerGroup)
    .channel(NioServerSocketChannel.class)
    .childHandler(new ChannelInitializer<NioSocketChannel>() {
        @Override
        protected void initChannel(NioSocketChannel ch) throws Exception {
            testInboundChannelPipeline(ch);
        }
    });
```

在ChannelInitializer类的initChannel()方法中，调用前页所展示的方法，构建管线，在本讲示例中，这条管线现在是这样的，包容三个入栈Handler

```
SimpleInHandlerA-->SimpleInHandlerB-->SimpleInHandlerC
```

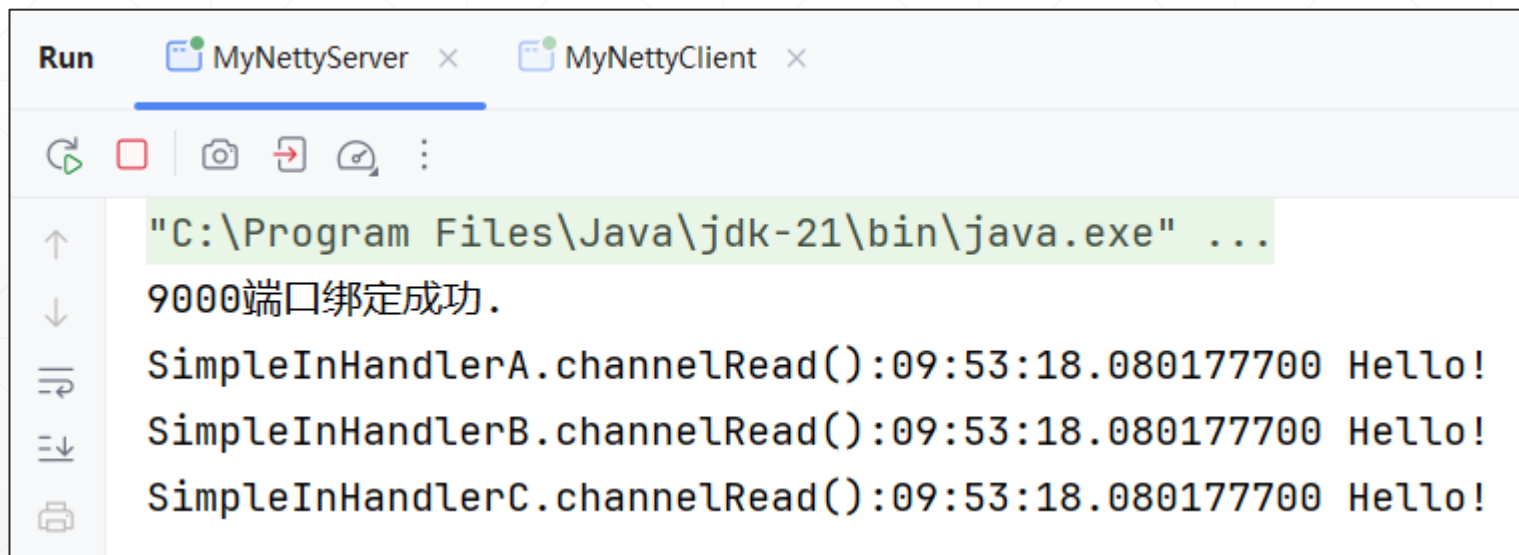
示例运行截图



```
Run  MyNettyServer x  MyNettyClient x
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
连接成功
发消息给Server:09:53:18.080177700 Hello!
```

客户端发给Server端消息

消息依次经过三个入站Handler

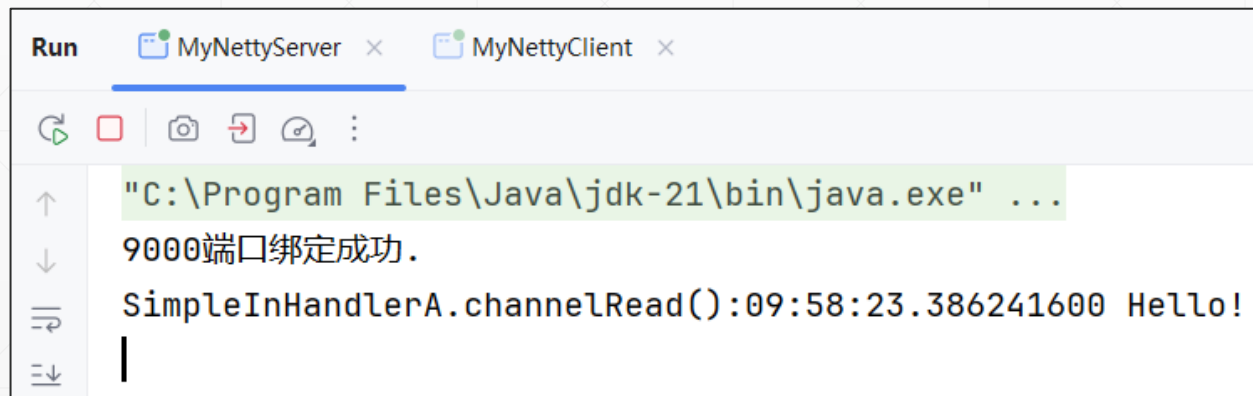


```
Run  MyNettyServer x  MyNettyClient x
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
9000端口绑定成功.
SimpleInHandlerA.channelRead():09:53:18.080177700 Hello!
SimpleInHandlerB.channelRead():09:53:18.080177700 Hello!
SimpleInHandlerC.channelRead():09:53:18.080177700 Hello!
```

仔细查看一下第一个入站Handler中的代码

```
public static class SimpleInHandlerA extends ChannelInboundHandlerAdapter {  
    @Override  
    public void channelRead(ChannelHandlerContext ctx, Object msg) throws Exception {  
        ByteBuf buf = (ByteBuf) msg;  
        String info = buf.toString(StandardCharsets.UTF_8);  
        System.out.println("SimpleInHandlerA.channelRead(): " + info);  
        ((ByteBuf) msg).resetReaderIndex();  
        //如果注释掉它, 则后继Handler将收不到消息  
        super.channelRead(ctx, msg);  
    }  
}
```

注意一下, 它调用了基类的channelRead()方法, 试着注释掉这句, 再次运行示例, 服务端会得到以下的结果:



```
Run  MyNettyServer x  MyNettyClient x  
"C:\Program Files\Java\jdk-21\bin\java.exe" ...  
9000端口绑定成功.  
SimpleInHandlerA.channelRead():09:58:23.386241600 Hello!  
|
```

可以看到, 现在后面的两个入站Handler, 都收不到消息了。

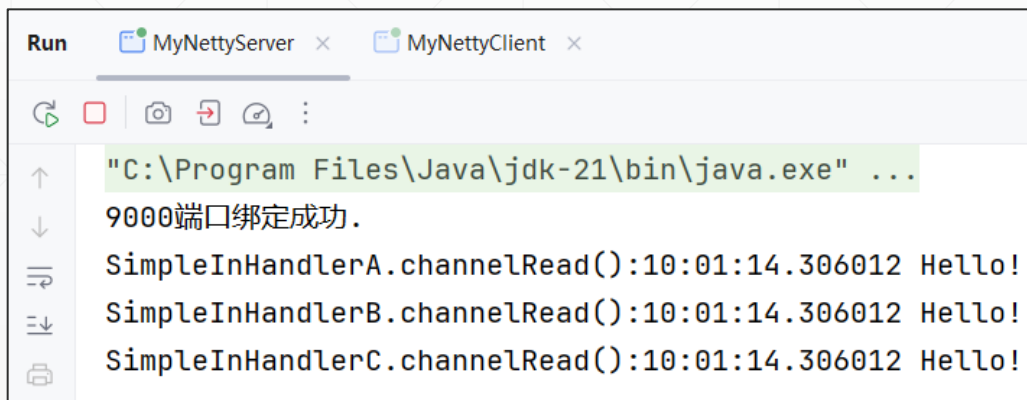
所以, 对基类同名方法的调用是很关键的。

另外一种转发消息的方式

```
public static class SimpleInHandlerA extends ChannelInboundHandlerAdapter {  
    @Override  
    public void channelRead(ChannelHandlerContext ctx, Object msg) throws Exception {  
        ByteBuf buf = (ByteBuf) msg;  
        String info = buf.toString(StandardCharsets.UTF_8);  
        System.out.println("SimpleInHandlerA.channelRead(): " + info);  
        ((ByteBuf) msg).resetReaderIndex();  
        //如果注释掉它, 则后继Handler将收不到消息  
        //super.channelRead(ctx, msg);  
        //手工恢复  
        ctx.fireChannelRead(msg);  
    }  
}
```

如果不调用基类的同名方法, 可以手工调用`fireChannelRead()`方法, 达到相同的效果。

↓
输出的结果又对了



```
Run  MyNettyServer x  MyNettyClient x  
"C:\Program Files\Java\jdk-21\bin\java.exe" ...  
9000端口绑定成功.  
SimpleInHandlerA.channelRead():10:01:14.306012 Hello!  
SimpleInHandlerB.channelRead():10:01:14.306012 Hello!  
SimpleInHandlerC.channelRead():10:01:14.306012 Hello!
```

事实上, `ChannelHandlerContext` 提供有一系列的 `fireXXX()` 方法, 主要功能就是触发“下一个” `Handler` 的相关事件, 把信息发给它, 这里所说的“触发事件”, 体现为下一个 `Handler` 的特定方法被回调。

关于出站处理器



ChannelHandler		
handlerRemoved(ChannelHandlerContext)		void
handlerAdded(ChannelHandlerContext)		void
exceptionCaught(ChannelHandlerContext, Throwable)		void

ChannelOutboundHandler		
read(ChannelHandlerContext)		void
bind(ChannelHandlerContext, SocketAddress, ChannelPromise)		void
write(ChannelHandlerContext, Object, ChannelPromise)		void
connect(ChannelHandlerContext, SocketAddress, SocketAddress, ChannelPromise)		void
close(ChannelHandlerContext, ChannelPromise)		void
flush(ChannelHandlerContext)		void
disconnect(ChannelHandlerContext, ChannelPromise)		void
deregister(ChannelHandlerContext, ChannelPromise)		void

出站处理器必须实现ChannelOutboundHandler接口，Netty提供ChannelOutboundHandlerAdapter这个“适配器（Adapter）类”，为接口中定义了默认实现，在实际开发中要自定义出站处理器时，可以从这个适配类派生，仅重写特定的方法即可。

再来看出站Handler的代码

与入站消息类似，示例中也定义了三个出栈Handler，名字为：SimpleOutHandlerA、SimpleOutHandlerB和SimpleOutHandlerC，示例如下：

```
public static class SimpleOutHandlerA extends ChannelOutboundHandlerAdapter {
    @Override
    public void write(ChannelHandlerContext ctx, Object msg, ChannelPromise promise) throws Exception {
        ByteBuf buf = (ByteBuf) msg;
        String info = "前一个发来: " + buf.toString(StandardCharsets.UTF_8) + " ";
        String nextInfo = "SimpleOutHandlerA: {" + info + "}";
        System.out.println("\nSimpleOutHandlerA.write()-->\n\t" + nextInfo);
        ByteBuf outBuf = Unpooled.wrappedBuffer(nextInfo.getBytes());
        //write()方法会导致消息向后一个出站Handler传送
        ctx.write(outBuf, promise);
        ctx.flush();
    }
}
```

这里的关键在于：write方法会把消息传给下一个Handler，而flush方法则通知Netty和操作系统，不需要等待缓冲区满，立即发送消息。

注意一下，示例在控制台输出了“是由哪个出站Handler发出的哪条消息”。

光有出站Handler，没有用~~~

对于Server端应用来说，如果你只定义了出站Handler并把它们装配为一条管线，你会发现“这条管线没用”，因为这条管线必须关联着一个Channel，并且这个Channel，必须启动一个“发送消息”的操作。

为此，示例定义了一个简单的入栈Handler，它先接收从客户端发来的消息，再使用write()方法启动管线的“出站”路线。

```
public static class SimpleInHandler extends ChannelInboundHandlerAdapter {  
  
    @Override  
    public void channelRead(ChannelHandlerContext ctx, Object msg) throws Exception {  
        ByteBuf buf = (ByteBuf) msg;  
        String info = "收到信息," + buf.toString(StandardCharsets.UTF_8);  
        System.out.println("SimpleInHandler.channelRead(): " + info);  
        String returnInfo = info + ",发送回执。";  
        ctx.write(Unpooled.copiedBuffer(returnInfo.getBytes()));  
    }  
}
```

现在可以装配管线

```
boot.group(bossGroup, workerGroup)
    .channel(NioServerSocketChannel.class)
    .childHandler(new ChannelInitializer<NioSocketChannel>() {
        @Override
        protected void initChannel(NioSocketChannel ch) throws Exception {
            //testInboundChannelPipeline(ch);
            testOutboundChannelPipeline(ch);
        }
    });
```



```
private static void testOutboundChannelPipeline(NioSocketChannel ch) {
    //顺序很重要, 出站Handler执行顺序: C-->B-->A
    ch.pipeline().addLast(new MyServerPipeline.SimpleOutHandlerA());
    ch.pipeline().addLast(new MyServerPipeline.SimpleOutHandlerB());
    ch.pipeline().addLast(new MyServerPipeline.SimpleOutHandlerC());
    //最后是进站Handler
    ch.pipeline().addLast(new MyServerPipeline.SimpleInHandler());
}
```

运行截图



```
Run  MyNettyServer x  MyNettyClient x
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
9000端口绑定成功.
SimpleInHandler.channelRead():收到信息,10:18:08.393290400 Hello!

SimpleOutHandlerC.write()-->
    SimpleOutHandlerC: (前一个发来"收到信息,10:18:08.393290400 Hello!,发送回执。")

SimpleOutHandlerB.write()-->
    SimpleOutHandlerB: [前一个发来:"SimpleOutHandlerC: (前一个发来"收到信息,10:18:08.393290400 Hello!,发送回执。")"]

SimpleOutHandlerA.write()-->
    SimpleOutHandlerA: {前一个发来:"SimpleOutHandlerB: [前一个发来:"SimpleOutHandlerC: (前一个发来"收到信息,10:18:08.393290400 Hello!,发送回执。")"]}]}
```

注意一下服务端管线中Handler的顺序是这样的：

SimpleOutHandlerA-->SimpleOutHandlerB-->SimpleOutHandlerC-->SimpleInHandler
而出站时，消息的传送路径则是C-->B-->A，正好与定义顺序相反。

另外要注意一下，入站Handler应该放在管线的末端，如果不这样安排，会发生什么呢？

进行试验，让入站Handler排在前面.....

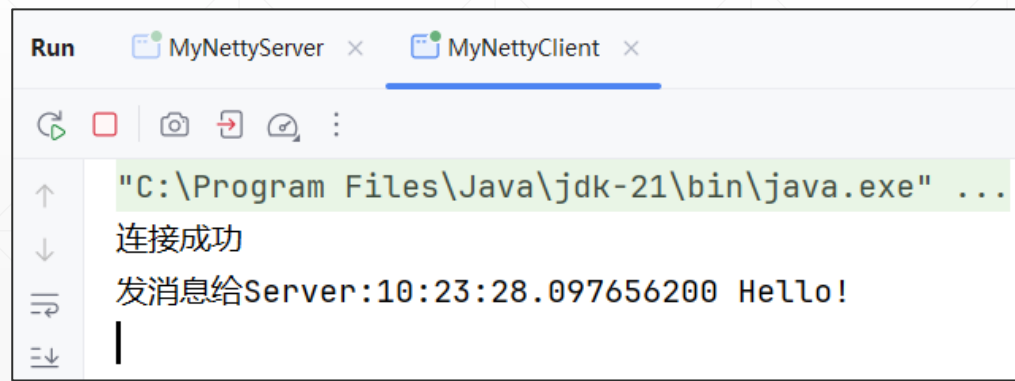
```
//测试出站pipeline
private static void testOutboundChannelPipeline(NioSocketChannel ch) {
    //入站Handler排在前面，会导致后面的出站Handler无法执行
    ch.pipeline().addLast(new MyServerPipeline.SimpleInHandler());
    //顺序很重要，出站Handler执行顺序：C-->B-->A
    ch.pipeline().addLast(new MyServerPipeline.SimpleOutHandlerA());
    ch.pipeline().addLast(new MyServerPipeline.SimpleOutHandlerB());
    ch.pipeline().addLast(new MyServerPipeline.SimpleOutHandlerC());
    //最后是入站Handler
    //ch.pipeline().addLast(new MyServerPipeline.SimpleInHandler());
}
```

从输出结果中可以看到，Server端的出站Handler，没一个收到了消息，相应地，客户端也没有能够收到任何服务端的回应！



The screenshot shows the console output of the MyNettyServer application. The text indicates that the 9000 port binding was successful and that the SimpleInHandler received a message from the client at 10:23:28.097656200, which was 'Hello!'.

```
Run  MyNettyServer x  MyNettyClient x
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
9000端口绑定成功.
SimpleInHandler.channelRead():收到信息,10:23:28.097656200 Hello!
```



The screenshot shows the console output of the MyNettyClient application. The text indicates that the connection to the server was successful and that a message 'Hello!' was sent to the server at 10:23:28.097656200.

```
Run  MyNettyServer x  MyNettyClient x
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
连接成功
发消息给Server:10:23:28.097656200 Hello!
```

结论



当构建管线时，Handler的顺序是非常重要的。



如果管线中同时包容有进站Handler和出站Handler，则应该把进站Handler放在管线后部。



如果管线中包容有多个进站Handler，则消息按照加入顺序流经各个Handler。



如果管线中包容有多个出站Handler，则消息按照加入顺序“**逆序**”流经各个Handler。